Keccak Circuit

개요

Keccak Circuit은 Keccak-256 해시 함수를 검증하는 회로입니다. 이 회로는 Ethereum에서 사용하는 표준 해시 함수인 Keccak-256의 모든 연산을 영지식 증명으로 검증하며, 바이트코드 해시, 트랜잭션 해시, 상태 트리 해시 등 다양한 곳에서 사용됩니다.

주요 역할:

- Keccak-f[1600] 순열 함수 구현 및 검증
- 임의 길이 입력에 대한 해시 계산
- Sparse word representation을 통한 효율적인 비트 연산
- Keccak Table을 통한 다른 회로와의 해시 결과 공유
- 멀티 블록 해싱 지원 (136바이트 단위 absorb)

파일 위치:

- 메인 파일: `zkevm-circuits/src/keccak_circuit.rs`
- 핵심 구현: `zkevm-circuits/src/keccak_circuit/keccak_packed_multi.rs`
- 셀 관리자: `zkevm-circuits/src/keccak_circuit/cell_manager.rs`
- 파라미터: `zkevm-circuits/src/keccak_circuit/param.rs`
- 유틸리티: `zkevm-circuits/src/keccak_circuit/util.rs`

Keccak-f[1600] 알고리즘

Keccak-256은 Keccak-f[1600] 순열 함수를 기반으로 합니다. 이 함수는 1600비트(200바이트) 상태에 대해 24라운드의 변환을 수행합니다.

상태 구조

Keccak 상태는 5×5 배열의 64비트 워드로 표현됩니다:

```
let mut s: [[F; 5]; 5]; // 25개의 64비트 워드
```

총 크기: 25 × 64 = 1600비트

라운드 함수

각 라운드는 5개의 단계로 구성됩니다:

1. **θ (Theta)**: 열(column) 패리티 계산 및 XOR
2. **ρ (Rho)**: 비트 회전
3. **π (Pi)**: 워드 순열
4. **χ (Chi)**: 비선형 변환
5. **ι (Iota)**: 라운드 상수 추가

라운드 상수:

```
pub const ROUND_CST: [u64; 25] = [
0x0000000000000001, 0x0000000000008082, 0x800000000000808a,
0x8000000080008000, 0x000000000000808b, 0x0000000080000001,
// ... 24개 라운드 상수
0x0000000000000000, // 25번째는 absorb 라운드용
];
```

Absorb와 Squeeze

Keccak은 스폰지 구조(sponge construction)를 사용합니다:

Absorb 단계:

- 입력 데이터를 136바이트(17개 워드) 블록으로 나눔
- 각 블록을 상태의 처음 17개 워드와 XOR
- 24라운드 순열 함수 적용

Squeeze 단계:

- 상태의 처음 32바이트(4개 워드)를 출력으로 추출
- Keccak-256의 경우 한 번의 squeeze로 충분

Rate와 Capacity:

```
pub const RATE: usize = 136; // 17 words × 8 bytes
pub const CAPACITY: usize = 64; // 8 words × 8 bytes
pub const NUM_WORDS_TO_ABSORB: usize = 17;
pub const NUM_WORDS_TO_SQUEEZE: usize = 4;
```

Sparse Word Representation

회로에서 비트 연산을 효율적으로 수행하기 위해 "sparse representation"을 사용합니다.

BIT_SIZE와 BIT_COUNT

```
pub const BIT_COUNT: usize = 3; // 각 비트를 표현하는 데 사용하는 비트 수
pub const BIT_SIZE: usize = 8; // 2^BIT_COUNT = 2^3 = 8
```

표현 방식

일반 64비트 워드를 sparse 형태로 변환:

일반 표현: 각 비트가 0 또는 1

```
예: 10000000 (128 in binary)
```

Sparse 표현: 각 비트를 3비트로 확장

```
예: |001|000|000|000|000|000|000|000|
MSB LSB
```

Pack과 Unpack

Pack 함수: 비트 배열을 sparse 워드로 변환

```
pub fn pack<F: Field>(bits: &[u8]) -> F {
pack_with_base(bits, BIT_SIZE)
}

pub fn pack_with_base<F: Field>(bits: &[u8], base: usize) -> F {
let base = F::from(base as u64);
bits.iter().rev().fold(F::zero(), |acc, &bit| {
acc * base + F::from(bit as u64)
})
```

```
}
```

Unpack 함수: Sparse 워드를 비트 배열로 변환

```
pub fn unpack<F: Field>(packed: F) -> [u8; 64] {
let mut bits = [0; 64];
let packed = Word::from_little_endian(packed.to_repr().as_ref());
let mask = Word::from(BIT_SIZE - 1); // 0b111
for (idx, bit) in bits.iter_mut().enumerate() {
*bit = ((packed >> (idx * BIT_COUNT)) & mask).as_u32() as u8;
}
bits
}
```

Sparse 연산의 장점

1. **XOR 연산**: 덧셈 후 정규화로 구현

```
// a XOR b = normalize(a + b)
// 예: 1 XOR 1 = normalize(1 + 1) = normalize(2) = 0
```

2. **AND 연산**: 곱셈으로 구현

```
// a AND b = a * b
// 예: 1 AND 1 = 1 * 1 = 1
```

3. **NOT 연산**: 1에서 빼기

```
// NOT a = 1 - a
```

4. **룩업 테이블 크기 감소**: 각 비트가 [0,1] 대신 [0,7] 범위를 가지므로 더 많은 비트를 한 번에 처리 가능

회로 구조

KeccakCircuitConfig

```
pub struct KeccakCircuitConfig<F> {
// 고정 컬럼 (selectors)
q_enable: Column<Fixed>, // 활성화 플래그
q_first: Column<Fixed>, // 첫 행
q_round: Column<Fixed>, // 라운드 실행
q_absorb: Column<Fixed>, // Absorb 단계
q_round_last: Column<Fixed>, // 마지막 라운드
q_padding: Column<Fixed>, // 패딩 처리
q_padding_last: Column<Fixed>, // 마지막 패딩
// Keccak Table
pub keccak_table: KeccakTable, // 다른 회로와 공유
// Cell Manager
pub cell_manager: CellManager<F>, // Advice 컬럼 관리
// 라운드 상수
```

```
round_cst: Column<Fixed>,

// 룩업 테이블

normalize_3: [TableColumn; 2], // 3으로 정규화 (XOR)

normalize_4: [TableColumn; 2], // 4로 정규화 (Rho/Pi)

normalize_6: [TableColumn; 2], // 6으로 정규화 (Theta)

chi_base_table: [TableColumn; 2], // Chi 연산

pack_table: [TableColumn; 2], // 바이트 패킹/언패킹

}
```

정규화 테이블

Sparse representation에서 XOR 연산은 덧셈 후 정규화로 구현됩니다.

Normalize_3 테이블: Absorb와 Iota에서 사용

```
입력 범위: [0, 2] (0+0, 0+1, 1+1)

출력: 입력 mod 2

0 -> 0, 1 -> 1, 2 -> 0
```

Normalize_4 테이블: Rho/Pi에서 사용

```
입력 범위: [0, 3]

출력: 입력 mod 2
```

Normalize_6 테이블: Theta에서 사용

```
입력 범위: [0, 5] (5개 워드의 합)
```

```
출력: 입력 mod 2
```

Chi Base 테이블

Chi 단계의 비선형 변환 `a ^ ((~b) & c)` 를 룩업으로 구현:

```
pub const CHI_BASE_LOOKUP_TABLE: [u8; 5] = [0, 1, 1, 0, 0];


// 계산: lookup[3 - 2*a + b - c]
// 예: a=1, b=1, c=0 -> lookup[3 - 2 - 1 + 1 + 0] = lookup[1] = 1
```

동작 원리:

```
a ^ ((~b) & c) = a ^ (NOT b AND c)


진리표:
a | b | c | ~b | (~b)&c | a^((~b)&c)
0 | 0 | 0 | 1 | 0 | 0
0 | 0 | 1 | 1 | 1 | 1
0 | 1 | 0 | 0 | 0 | 0
0 | 1 | 1 | 0 | 0 | 0
1 | 0 | 0 | 1 | 0 | 1
1 | 0 | 1 | 1 | 1 | 0
1 | 1 | 0 | 0 | 0 | 1
```

```
1 | 1 | 1 | 0 | 0 | 1
```

Pack 테이블

바이트와 sparse 표현 간 변환:

```
입력: sparse 워드 (8비트 × BIT_SIZE)
출력: 일반 바이트 (0~255)

예: |001|000|000|000|000|000|000|000| -> 128
```

CellManager (Keccak 전용)

Keccak Circuit은 특별한 CellManager를 사용하여 셀을 수직으로 할당합니다.

수직 할당 전략

EVM Circuit과 달리 Keccak은 **수직 정렬**을 사용합니다:

```
Row 0: [cell_0_0] [cell_1_0] [cell_2_0] ...
Row 1: [cell_0_1] [cell_1_1] [cell_2_1] ...
Row 2: [cell_0_2] [cell_1_2] [cell_2_2] ...
...
Row N: [cell_0_N] [cell_1_N] [cell_2_N] ...
```

CellManager 구조

```rust
pub struct CellManager<F> {
height: usize, // 라운드당 행 수
columns: Vec<CellColumn<F>>, // Advice 컬럼들
rows: Vec<usize>, // 각 행의 현재 컬럼 위치
num_unused_cells: usize, // 사용하지 않은 셀 수
}
```

주요 메서드

query_cell(): 다음 사용 가능한 셀 할당

```rust
pub fn query_cell(&mut self, meta: &mut ConstraintSystem<F>) -> Cell<F> {
let (row_idx, column_idx) = self.get_position();
// 가장 적게 사용된 행에 셀 할당
self.query_cell_at_pos(meta, row_idx as i32, column_idx)
}
```

query_cell_at_row(): 특정 행에 셀 할당

```rust
pub fn query_cell_at_row(&mut self, meta: &mut ConstraintSystem<F>, row_idx:
let column_idx = self.rows[row_idx as usize];
self.rows[row_idx as usize] += 1;
self.query_cell_at_pos(meta, row_idx, column_idx)
```

```
}
```

start_region(): 새로운 영역 시작

```
pub fn start_region(&mut self) -> usize {
// 모든 행을 같은 컬럼에서 시작하도록 정렬
let width = self.get_width();
for row in self.rows.iter_mut() {
self.num_unused_cells += width - *row;
*row = width;
}
width
}
```

영역(Region) 개념

CellManager는 회로를 여러 영역으로 나눕니다:

1. **상태 변수 영역**: 25개 상태 워드 + absorb 변수
2. **Absorb 영역**: Absorb 연산 처리
3. **입력 바이트 영역**: 입력 데이터 언패킹
4. **패딩 영역**: 패딩 플래그
5. **Theta 영역**: Theta 단계 계산
6. **Rho/Pi 영역**: Rho/Pi 단계 계산
7. **Chi 영역**: Chi 단계 계산

8. **Iota 영역**: Iota 단계 계산

9. **Squeeze 영역**: 출력 데이터 처리

각 영역은 `start_region()` 으로 구분되며, 이를 통해 컬럼 재사용을 최적화합니다.

라운드별 단계 구현

θ (Theta) 단계

목적: 각 열의 패리티를 계산하고 상태에 XOR

알고리즘:

```
// 1. 각 열의 합 계산
for i in 0..5 {
c[i] = s[i][0] + s[i][1] + s[i][2] + s[i][3] + s[i][4];
}

// 2. 정규화 (XOR 구현)
bc[i] = normalize(c[i]);

// 3. 회전 및 XOR
for i in 0..5 {
t[i] = bc[(i + 4) % 5] + rotate(bc[(i + 1) % 5], 1);
for j in 0..5 {
s[i][j] = s[i][j] + t[i];
}
```

```
}
```

회로 구현:

```
// Part size 결정
let part_size_c = get_num_bits_per_theta_c_lookup(); // 보통 6~7

// 1. c 계산 및 분할
let mut c_parts = Vec::new();
for s in s.iter() {
let c = s[0] + s[1] + s[2] + s[3] + s[4];
c_parts.push(split::expr(meta, &mut cell_manager, &mut cb, c, 1, part_size_c))
}

// 2. bc 정규화 (normalize_6 테이블 사용)
let mut bc = Vec::new();
for c in c_parts {
bc.push(transform::expr(
"theta c",
meta,
&mut cell_manager,
&mut lookup_counter,
c,
```

```
normalize_6,
true,
));
}

// 3. t 계산 및 상태 업데이트
for i in 0..5 {
let t = decode::expr(bc[(i + 4) % 5].clone())
+ decode::expr(rotate(bc[(i + 1) % 5].clone(), 1, part_size_c));
for j in 0..5 {
os[i][j] = s[i][j].clone() + t.clone();
}
}
```

핵심 포인트:

- `c` 는 최대 5개 워드의 합이므로 각 비트가 [0,5] 범위
- `normalize_6` 테이블로 [0,5] → [0,1] 변환
- 회전은 part 단위로 수행하여 효율성 향상

ρ (Rho) 단계

목적: 각 워드를 지정된 비트만큼 회전

회전 행렬:

```
pub const RHO_MATRIX: [[usize; 5]; 5] = [
```

```
[0, 36, 3, 41, 18],
[1, 44, 10, 45, 2],
[62, 6, 43, 15, 61],
[28, 55, 25, 21, 56],
[27, 20, 39, 8, 14],
];
```

예: `s[1][2]` 는 10비트 왼쪽 회전

π (Pi) 단계

목적: 워드 위치 순열

알고리즘:

```
for i in 0..5 {
for j in 0..5 {
s_new[j][(2*i + 3*j) % 5] = s[i][j];
}
}
```

Rho/Pi 결합 구현

Rho와 Pi는 효율성을 위해 결합되어 구현됩니다:

```
// Rho/Pi를 한 번에 수행
for j in 0..5 {
```

```
for i in 0..5 {
// s[i][j]를 RHO_MATRIX[i][j]만큼 회전하여
// s_new[j][(2*i + 3*j) % 5]에 저장
let s_parts = split_uniform::expr(
meta,
&rho_pi_chi_cells[0][j][(2 * i + 3 * j) % 5],
&mut cell_manager,
&mut cb,
s[i][j].clone(),
RHO_MATRIX[i][j],
part_size,
);
let s_parts = transform_to::expr(
"rho/pi",
meta,
&rho_pi_chi_cells[1][j][(2 * i + 3 * j) % 5],
&mut lookup_counter,
s_parts.clone(),
normalize_4,
true,
);
os_part[(2 * i + 3 * j) % 5] = s_parts.clone();
}
```

```
}
```

split_uniform의 역할:

- 회전된 워드를 균일한 part 크기로 분할
- 모든 워드의 part가 같은 행에 위치하도록 보장
- Chi 단계에서 효율적인 처리를 위한 준비

χ (Chi) 단계

목적: 비선형 변환 적용

알고리즘:

```
for i in 0..5 {
for j in 0..5 {
s[i][j] = s[i][j] ^ ((~s[(i+1)%5][j]) & s[(i+2)%5][j]);
}
}
```

회로 구현:

```
// Chi는 5개 컬럼씩 그룹으로 처리
for idx in 0..num_columns {
for i in 0..5 {
// input = 3 - 2*a + b - c
// 여기서 a=s[i][j], b=s[(i+1)%5][j], c=s[(i+2)%5][j]
let input = scatter::expr(3, part_size_base)
```

```
- 2.expr() * input[i].clone()
+ input[(i + 1) % 5].clone()
- input[(i + 2) % 5].clone();
let  output = output[i].clone();
// Chi base 테이블 룩업
meta.lookup("chi base", |_| {
vec![
(input.clone(), chi_base_table[0]),
(output.clone(), chi_base_table[1]),
]
});
}
}
```

핵심 포인트:

- 각 part를 독립적으로 처리
- 5개 워드가 같은 행에 있어야 함 (Rho/Pi에서 준비)
- `scatter::expr(3, part_size)` 는 모든 비트가 3인 packed 워드 생성

ι (Iota) 단계

목적: 라운드 상수 추가

알고리즘:

```
s[0][0] = s[0][0] ^ ROUND_CST[round];
```

회로 구현:

```
// 라운드 상수를 sparse 형태로 변환
let round_cst_expr = meta.query_fixed(round_cst, Rotation::cur());


// s[0][0]에 라운드 상수 추가
let part_size = get_num_bits_per_absorb_lookup();
let input = s[0][0].clone() + round_cst_expr.clone();
let iota_parts = split::expr(meta, &mut cell_manager, &mut cb, input, 0, part


// 정규화 (XOR 구현)
s[0][0] = decode::expr(transform::expr(
"iota",
meta,
&mut cell_manager,
&mut lookup_counter,
iota_parts,
normalize_3,
true,
));
```

Absorb 프로세스

Absorb 위치

상태의 처음 17개 워드(136바이트)가 absorb에 사용됩니다:

```
pub fn get_absorb_positions() -> Vec<(usize, usize)> {
let mut absorb_positions = Vec::new();
for j in 0..5 {
for i in 0..5 {
if i + j * 5 < 17 {
absorb_positions.push((i, j));
}
}
}
absorb_positions
}
// 결과: [(0,0), (1,0), (2,0), (3,0), (4,0),
// (0,1), (1,1), (2,1), (3,1), (4,1),
// (0,2), (1,2), (2,2), (3,2), (4,2),
// (0,3), (1,3)]
```

Absorb 데이터 구조

```
pub struct AbsorbData<F: Field> {
pub from: F, // 현재 상태 값
```

```
pub absorb: F, // 입력 데이터
pub result: F, // from XOR absorb
}
```

Absorb 구현

24라운드에 걸쳐 분산 처리:

- 각 라운드에서 1개 워드(8바이트)씩 absorb
- 처음 17개 라운드에서만 실제 데이터 처리
- 나머지 7개 라운드는 absorb 없이 순열만 수행

```
// Absorb 게이트
meta.create_gate("absorb", |meta| {
let continue_hash = not::expr(start_new_hash(meta, Rotation::cur()));
let absorb_positions = get_absorb_positions();
let mut a_slice = 0;
for j in 0..5 {
for i in 0..5 {
if absorb_positions.contains(&(i, j)) {
// 이전 해시가 계속되는 경우, 이전 상태 검증
cb.condition(continue_hash.clone(), |cb| {
cb.require_equal(
"absorb verify input",
absorb_from_next[a_slice].clone(),
```

```
pre_s[i][j].clone(),
);
});
// 결과 = 이전 상태 XOR 입력 데이터
cb.require_equal(
"absorb result copy",
select::expr(
continue_hash.clone(),
absorb_result_next[a_slice].clone(),
absorb_data_next[a_slice].clone(),
),
s_next[i][j].clone(),
);
a_slice += 1;
} else {
// Absorb 위치가 아니면 상태 그대로 복사
cb.require_equal(
"absorb state copy",
pre_s[i][j].clone() * continue_hash.clone(),
s_next[i][j].clone(),
);
}
}
```

```
}
cb.gate(meta.query_fixed(q_absorb, Rotation::cur()))
});
```

입력 데이터 처리

각 라운드에서 8바이트 워드를 개별 바이트로 언패킹:

```
// 워드를 8개 바이트로 분할
let packed_parts = split::expr(
meta,
&mut cell_manager,
&mut cb,
absorb_data.expr(),
0,
NUM_BYTES_PER_WORD, // 8
);

// Pack 테이블로 언패킹 (역방향)
let input_bytes = transform::expr(
"input unpack",
meta,
&mut cell_manager,
&mut lookup_counter,
```

```
packed_parts,
pack_table.into_iter().rev().collect::<Vec<_>>().try_into().unwrap(),
true,
);
```

Squeeze 프로세스

Squeeze 데이터 구조

```
pub struct SqueezeData<F: Field> {
packed: F, // 출력할 워드
}
```

Squeeze 구현

24라운드 후 4개 워드 추출:

```
// Squeeze 게이트
meta.create_gate("squeeze", |meta| {
let start_new_hash = start_new_hash(meta, Rotation::cur());
// 출력할 4개 워드
let hash_words: Vec<_> = pre_s
.into_iter()
.take(4)
.map(|a| a[0].clone())
```

```
.collect();
// 이전 행에서 워드를 바이트로 변환했는지 검증
for (idx, word) in  hash_words.iter().enumerate() {
cb.condition(start_new_hash.clone(), |cb| {
cb.require_equal(
"squeeze verify packed",
word.clone(),
squeeze_from_prev[idx].clone(),
);
});
}
// 해시 바이트들을 RLC로 결합
let  hash_bytes_le = hash_bytes.into_iter().rev().collect::<Vec<_>>();
let  rlc = compose_rlc::expr(&hash_bytes_le, challenges.evm_word());
cb.condition(start_new_hash, |cb| {
cb.require_equal(
"hash rlc check",
rlc,
meta.query_advice(hash_rlc, Rotation::cur()),
);
});
cb.gate(meta.query_fixed(q_round_last, Rotation::cur()))
```

```
});
```

Squeeze 타이밍:

- 마지막 4개 라운드에서 각 라운드마다 1개 워드씩 바이트로 변환
- 최종 라운드(q_round_last)에서 모든 바이트를 RLC로 결합하여 검증

패딩 규칙

Keccak은 입력 데이터에 패딩을 추가하여 136바이트의 배수로 만듭니다.

패딩 알고리즘

```
// 1. 비트 배열로 변환
let mut bits = into_bits(bytes);

// 2. 첫 패딩 비트 추가 (1)
bits.push(1);

// 3. 중간 패딩 (0으로 채움)
while (bits.len() + 1) % RATE_IN_BITS != 0 {
bits.push(0);
}

// 4. 마지막 패딩 비트 추가 (1)
```

```
bits.push(1);
```

예시:

- 입력: `[0x01, 0x02]` (16비트)
- 패딩 후: `[0x01, 0x02, 0x01, 0x00, ..., 0x80]`
- 첫 패딩 바이트: `0x01` (비트 1)
- 중간: `0x00` 으로 채움
- 마지막 바이트: `0x80` (비트 1이 MSB 위치)

패딩 플래그

회로는 각 바이트에 대해 패딩 여부를 추적합니다:

```
// 패딩 플래그 셀
let mut is_paddings = Vec::new();
for _ in input_bytes.iter() {
is_paddings.push(cell_manager.query_cell(meta));
}
```

패딩 제약 조건:

```
meta.create_gate("padding", |meta| {
let q_padding = meta.query_fixed(q_padding, Rotation::cur());
let q_padding_last = meta.query_fixed(q_padding_last, Rotation::cur());
// 모든 패딩 플래그는 boolean
for is_padding in is_paddings.iter() {
```

```
cb.condition(meta.query_fixed(q_enable, Rotation::cur()), |cb| {
cb.require_boolean("is_padding boolean", is_padding.expr());
});
}
// 패딩 전환은 한 번만 (0 -> 1)
for  idx  in  0..is_paddings.len() {
let  is_padding_prev = if  idx == 0 {
prev_is_padding.expr()
} else {
is_paddings[idx - 1].expr()
};
let  is_first_padding = is_paddings[idx].expr() - is_padding_prev.clone();
cb.condition(q_padding.expr(), |cb| {
cb.require_boolean("padding step boolean", is_first_padding.clone());
});
// 패딩 바이트 값 검증
if  idx == is_paddings.len() - 1 {
// 마지막 바이트: 128 또는 129 (첫 패딩이면 129)
cb.condition(
and::expr([q_padding_last.expr(), is_paddings[idx].expr()]),
|cb| {
cb.require_equal(
"padding start/end byte",
```

```
input_bytes[idx].expr.clone(),
is_first_padding.expr() + 128.expr(),
);
},
);
} else {
// 중간 바이트: 0 또는 1 (첫 패딩이면 1)
cb.condition(
and::expr([q_padding.expr(), is_paddings[idx].expr()]),
|cb| {
cb.require_equal(
"padding start/intermediate byte",
input_bytes[idx].expr.clone(),
is_first_padding.expr(),
);
},
);
}
}
cb.gate(1.expr())
});
```

RLC 계산

입력 데이터 RLC

입력 데이터는 RLC로 압축되어 Keccak Table에 저장됩니다:

```
// 각 바이트마다 RLC 누적
data_rlc = data_rlc * challenge + byte_value


// 패딩 바이트는 제외
if !is_padding {
data_rlc = data_rlc * challenge + byte_value;
}
```

RLC 제약 조건:

```
meta.create_gate("length and data rlc", |meta| {
let  q_padding = meta.query_fixed(q_padding, Rotation::cur());
// 길이 업데이트
cb.condition(q_padding.expr(), |cb| {
cb.require_equal(
"update length",
length.clone(),
length_prev.clone() * not::expr(start_new_hash_prev.expr())
+ sum::expr(
```

```
is_paddings.iter().map(|is_padding| not::expr(is_padding.expr())),
),
);
// 초기 data_rlc (새 해시면 0, 아니면 이전 값)
let  data_rlc_zero_or_prev = data_rlc_prev.clone() * not::expr(start_new_hash_pr
cb.require_equal("initial data rlc", data_rlc_zero_or_prev, new_data_rlc.clone()
// 각 바이트 처리
for (idx, (byte, is_padding)) in  input_bytes.iter().zip(is_paddings.iter()).enu
new_data_rlc = select::expr(
is_padding.expr(),
new_data_rlc.clone(), // 패딩이면 그대로
new_data_rlc.clone() * challenges.keccak_input() + byte.expr.clone(),
);
let  data_rlc_after_this_byte = data_rlcs[NUM_BYTES_PER_WORD - (idx + 1)].expr()
cb.require_equal("intermediate data rlc", data_rlc_after_this_byte.clone(), new_
new_data_rlc = data_rlc_after_this_byte;
}
});
cb.gate(1.expr())
});
```

출력 해시 RLC

출력 해시(32바이트)도 RLC로 압축됩니다:

```
// 4개 워드 = 32바이트를 little-endian으로 변환
let hash_bytes_le = s
.into_iter()
.take(4)
.flat_map(|a| to_bytes::value(&unpack(a[0])))
.rev()
.collect::<Vec<_>>();

// RLC 계산
hash_rlc = challenges.evm_word().map(|challenge_value| {
rlc::value(&hash_bytes_le, challenge_value)
});
```

멀티 블록 처리

블록 체이닝

입력이 136바이트보다 크면 여러 블록으로 나뉩니다:

```
let chunks = bits.chunks(RATE_IN_BITS); // 136바이트 = 1088비트
for (idx, chunk) in chunks.enumerate() {
let is_final_block = idx == num_chunks - 1;
// 각 블록마다 24라운드 수행
```

```
// 상태는 블록 간 유지됨
}
```

상태 연속성:

- 첫 블록: 상태를 0으로 초기화
- 중간 블록: 이전 블록의 최종 상태 사용
- 마지막 블록: `is_final = true` 설정

is_final 플래그

```
meta.create_gate("is final", |meta| {
// Absorb 행에서 is_final 검증
cb.condition(
meta.query_fixed(q_absorb, Rotation::cur())
- meta.query_fixed(q_first, Rotation::cur()),
|cb| {
cb.require_equal(
"is_final needs to be the same as the last is_padding in the block",
meta.query_advice(is_final, Rotation::cur()),
last_is_padding_in_block.expr(),
);
},
);
// 라운드의 첫 행에서만 is_final이 1일 수 있음
```

```
cb.condition(
(1..get_num_rows_per_round() as i32)
.map(|i| meta.query_fixed(q_enable, Rotation(-i)))
.fold(0.expr(), |acc, elem| acc + elem),
|cb| {
cb.require_zero(
"is_final only when q_enable",
meta.query_advice(is_final, Rotation::cur()),
);
},
);
cb.gate(1.expr())
});
```

KeccakRow 구조

각 행의 witness 데이터:

```
pub struct KeccakRow<F: Field> {
pub q_enable: bool, // 행 활성화
pub q_round: bool, // 라운드 실행
pub q_absorb: bool, // Absorb 단계
pub q_round_last: bool, // 마지막 라운드
```

```
pub q_padding: bool, // 패딩 처리
pub q_padding_last: bool, // 마지막 패딩
pub round_cst: F, // 라운드 상수
pub is_final: bool, // 마지막 블록
pub length: usize, // 입력 길이
pub data_rlc: Value<F>, // 입력 RLC
pub hash_rlc: Value<F>, // 출력 RLC
pub cell_values: Vec<F>, // 모든 셀 값
}
```

Witness 생성

multi_keccak 함수

여러 입력에 대한 Keccak witness 생성:

```
pub fn multi_keccak<F: Field>(
bytes: &[Vec<u8>],
challenges: Challenges<Value<F>>,
capacity: Option<usize>,
) -> Result<Vec<KeccakRow<F>>, Error> {
let mut rows: Vec<KeccakRow<F>> = Vec::new();
// 더미 첫 행 추가
for idx in 0..get_num_rows_per_round() {
```

```
rows.push(KeccakRow {
q_enable: idx == 0,
q_absorb: idx == 0,
is_final: false,
length: 0,
data_rlc: Value::known(F::zero()),
hash_rlc: Value::known(F::zero()),
cell_values: Vec::new(),
// ...
});
}
// 각 입력에 대해 병렬로 처리
let real_rows: Vec<_> = bytes
.par_iter()
.flat_map_iter(|bytes| keccak_rows(bytes, challenges))
.collect();
rows.extend(real_rows);
// 용량에 맞게 패딩
if let Some(capacity) = capacity {
let padding_rows = {
let mut rows = Vec::new();
keccak(&mut rows, &[], challenges); // 빈 입력
rows
```

```
};
while rows.len() < (1 + capacity * (NUM_ROUNDS + 1)) * get_num_rows_per_round()
rows.extend(padding_rows.clone());
}
if rows.len() > (1 + capacity * (NUM_ROUNDS + 1)) * get_num_rows_per_round() {
return Err(Error::BoundsFailure);
}
}
Ok(rows)
}
```

keccak 함수

단일 입력에 대한 처리:

```
pub fn keccak<F: Field>(
rows: &mut Vec<KeccakRow<F>>,
bytes: &[u8],
challenges: Challenges<Value<F>>,
) {
let mut bits = into_bits(bytes);
let mut s = [[F::zero(); 5]; 5];
// 패딩 추가
bits.push(1);
```

```
while (bits.len() + 1) % RATE_IN_BITS != 0 {
bits.push(0);
}
bits.push(1);
let chunks = bits.chunks(RATE_IN_BITS);
let num_chunks = chunks.len();
for (idx, chunk) in chunks.enumerate() {
let is_final_block = idx == num_chunks - 1;
// Absorb 데이터 준비
let mut absorb_rows = Vec::new();
for (idx, &(i, j)) in get_absorb_positions().iter().enumerate() {
let absorb = pack(&chunk[idx * 64..(idx + 1) * 64]);
let from = s[i][j];
s[i][j] = field_xor(s[i][j], absorb);
absorb_rows.push(AbsorbData { from, absorb, result: s[i][j] });
}
// 25개 라운드 (24 + 1 absorb)
for round in 0..NUM_ROUNDS + 1 {
let mut cell_manager = CellManager::new(get_num_rows_per_round());
let mut region = KeccakRegion::new();
// 상태 할당
for s in &s {
for s in s {
```

```
let cell = cell_manager.query_cell_value();
cell.assign(&mut region, 0, *s);
}
}
// Absorb 데이터 할당
// ... (absorb_from, absorb_data, absorb_result)
if round != NUM_ROUNDS {
// Theta, Rho/Pi, Chi, Iota 수행
// ...
}
// 행 생성
for row_idx in 0..get_num_rows_per_round() {
rows.push(KeccakRow {
q_enable: row_idx == 0,
q_round: row_idx == 0 && round < NUM_ROUNDS,
q_absorb: row_idx == 0 && round == NUM_ROUNDS,
q_round_last: row_idx == 0 && round == NUM_ROUNDS,
q_padding: row_idx == 0 && round < NUM_WORDS_TO_ABSORB,
q_padding_last: row_idx == 0 && round == NUM_WORDS_TO_ABSORB - 1,
round_cst: pack_u64(ROUND_CST[round]),
is_final: is_final_block && round == NUM_ROUNDS && row_idx == 0,
length: round_lengths[round],
data_rlc: round_data_rlcs[round][row_idx],
```

```
hash_rlc,
cell_values: regions[round].rows[row_idx].clone(),
});
}
}
}
}
```

Keccak Table

테이블 스키마

```
pub struct KeccakTable {
pub q_enable: Column<Fixed>, // 활성화
pub is_final: Column<Advice>, // 마지막 블록
pub input_rlc: Column<Advice>, // 입력 RLC
pub input_len: Column<Advice>, // 입력 길이
pub output_rlc: Column<Advice>, // 출력 RLC (해시)
}
```

테이블 사용

다른 회로에서 Keccak 해시를 검증할 때 사용:

```
// Bytecode Circuit에서 코드 해시 검증
```

```
meta.lookup_any("keccak256_table_lookup", |meta| {
let  enable = is_byte_to_header(meta);
vec![
(enable.clone(), keccak_table.is_final),
(enable.clone() * value_rlc, keccak_table.input_rlc),
(enable.clone() * length, keccak_table.input_len),
(enable * code_hash, keccak_table.output_rlc),
]
});
```

최적화 기법

1. Sparse Word Representation

장점:

- XOR을 덧셈 + 정규화로 구현 (degree 감소)
- 여러 비트를 한 번에 처리 (룩업 횟수 감소)
- 비트 연산을 필드 연산으로 변환

트레이드오프:

- 각 워드가 더 많은 공간 차지 (64비트 → 192비트)
- 정규화 룩업 필요

2. 동적 Part Size

환경 변수로 part size 조정 가능:

```
pub fn get_num_bits_per_lookup(range: usize) -> usize {
let log_height = get_degree(); // KECCAK_DEGREE 환경 변수
let height = 2usize.pow(log_height as u32);
let num_unusable_rows = keccak_unusable_rows();
let mut num_bits = 1;
while range.pow(num_bits + 1) + num_unusable_rows <= height {
num_bits += 1;
}
num_bits
}
```

예시:

- `KECCAK_DEGREE=19` : absorb part_size=11, theta part_size=7
- `KECCAK_DEGREE=20` : 더 큰 part size 가능

3. 라운드당 행 수 조정

```
pub fn get_num_rows_per_round() -> usize {
let r = var("KECCAK_ROWS")
.unwrap_or_else(|_| format!("{DEFAULT_KECCAK_ROWS}"))
.parse()
.expect("Cannot parse KECCAK_ROWS");
assert!(r > NUM_BYTES_PER_WORD);
r
```

```
}
```

트레이드오프:

- 행 수 증가: 더 많은 컬럼 재사용 가능, 회로 높이 증가
- 행 수 감소: 컬럼 수 증가, 회로 높이 감소

4. 영역 기반 컬럼 재사용

`start_region()` 을 통해 컬럼을 여러 영역에서 재사용:

```
// 영역 1: 상태 변수 (25개 워드)
for i in 0..5 {
for j in 0..5 {
let cell = cell_manager.query_cell(meta);
s[i][j] = cell.expr();
}
}

// 영역 2: Absorb 계산 (컬럼 재사용)
cell_manager.start_region();
let absorb_fat = split::expr(...);

// 영역 3: Theta 계산 (컬럼 재사용)
cell_manager.start_region();
```

```
let c_parts = ...;
```

5. Rho/Pi/Chi 결합

세 단계를 결합하여 중간 결과 저장 최소화:

- Rho/Pi 출력을 Chi 입력으로 직접 사용
- 5개 워드를 같은 행에 배치하여 Chi 효율화
- 중간 디코딩/인코딩 과정 제거

6. 병렬 Witness 생성

```
let real_rows: Vec<_> = bytes
.par_iter() // Rayon 병렬 처리
.flat_map_iter(|bytes| keccak_rows(bytes, challenges))
.collect();
```

여러 입력에 대한 witness를 병렬로 생성하여 성능 향상

회로 파라미터

기본 설정

```
pub const DEFAULT_KECCAK_ROWS: usize = 12;
pub const MAX_DEGREE: usize = 9;
pub const NUM_ROUNDS: usize = 24;
pub const NUM_WORDS_TO_ABSORB: usize = 17;
```

```rust
pub const NUM_WORDS_TO_SQUEEZE: usize = 4;
```

룩업 범위

```rust
pub const ABSORB_LOOKUP_RANGE: usize = 3; // [0, 2]
pub const THETA_C_LOOKUP_RANGE: usize = 6; // [0, 5]
pub const RHO_PI_LOOKUP_RANGE: usize = 4; // [0, 3]
pub const CHI_BASE_LOOKUP_RANGE: usize = 5; // [0, 4]
```

용량 계산

```rust
pub fn capacity_for_row(num_rows: usize) -> Option<usize> {
if num_rows > 0 {
// 사용 불가능한 행 제외
Some(num_rows / ((NUM_ROUNDS + 1) * get_num_rows_per_round()) - 2)
} else {
None
}
}
```

예시:

- `num_rows = 2^20`, `KECCAK_ROWS = 12`
- 용량 = 1048576 / (25 × 12) - 2 = 3493개 해시

구현 예시

간단한 해시 계산

```
// 입력: "hello"
let input = b"hello";
let challenges = Challenges::mock(Value::known(F::from(123)));

// Witness 생성
let rows = keccak_rows(input, challenges);

// 결과:
// - 1개 블록 (5바이트 < 136바이트)
// - 25개 라운드 × 12행 = 300행
// - is_final = true (마지막 블록)
```

멀티 블록 해시

```
// 입력: 200바이트 (136 + 64)
let input = vec![0u8; 200];

// 2개 블록 필요:
// - 블록 1: 136바이트 (is_final = false)
// - 블록 2: 64바이트 + 패딩 (is_final = true)
```

```
// 총 행 수: 2 × 25 × 12 = 600행
```

여러 입력 처리

```
let inputs = vec![
b"hello".to_vec(),
b"world".to_vec(),
vec![0u8; 200],
];

let rows = multi_keccak(&inputs, challenges, Some(100));

// 결과:
// - 더미 행: 12행
// - "hello": 300행
// - "world": 300행
// - 200바이트: 600행
// - 패딩: 나머지를 빈 해시로 채움
```

제약 조건 요약

---

1. 상태 전환

```
// 다음 라운드의 상태 = 현재 라운드 계산 결과
for i in 0..5 {
for j in 0..5 {
s_next[i][j] == computed_s[i][j]
}
}
```

2. Absorb 검증

```
// Absorb 위치: 이전 상태 XOR 입력 데이터
if is_absorb_position(i, j) {
if continue_hash {
absorb_from == pre_s[i][j]
}
s_next[i][j] == select(continue_hash, absorb_result, absorb_data)
}

// 비 Absorb 위치: 상태 유지
if !is_absorb_position(i, j) {
s_next[i][j] == pre_s[i][j] * continue_hash
}
```

3. 패딩 검증

```
// 패딩 플래그는 boolean
is_padding * (1 - is_padding) == 0


// 패딩 전환은 한 번만
(is_padding - is_padding_prev) * (1 - is_padding + is_padding_prev) == 0


// 첫 패딩 바이트: 1
if  is_first_padding && !is_last_byte {
input_byte == 1
}


// 마지막 패딩 바이트: 128 또는 129
if  is_first_padding && is_last_byte {
input_byte == 129
} else  if  is_last_byte {
input_byte == 128
}


// 중간 패딩 바이트: 0
if  is_padding && !is_first_padding && !is_last_byte {
input_byte == 0
}
```

4. 길이 및 RLC 업데이트

```
// 새 해시 시작 시 길이 초기화
if  start_new_hash {
length == 0
data_rlc == 0
}

// 패딩이 아닌 바이트 개수만큼 길이 증가
length_next == length + count_non_padding_bytes

// RLC 누적 (패딩 제외)
for  each  byte {
if !is_padding {
data_rlc_next == data_rlc * challenge + byte
} else {
data_rlc_next == data_rlc
}
}
```

5. 출력 검증

```
// 마지막 라운드에서 해시 RLC 검증
```

```
if  q_round_last && is_final {
hash_rlc == rlc(hash_bytes, challenge)
}

// Squeeze 워드 검증
for  i  in  0..4 {
squeeze_from_prev[i] == s[i][0]
}
```

6. 정규화 룩업

```
// Absorb/Iota: normalize_3
lookup(input, output) in  normalize_3_table
where  output == input  mod  2, input  in [0, 2]

// Rho/Pi: normalize_4
lookup(input, output) in  normalize_4_table
where  output == input  mod  2, input  in [0, 3]

// Theta: normalize_6
lookup(input, output) in  normalize_6_table
where  output == input  mod  2, input  in [0, 5]
```

7. Chi 룩업

```
// Chi base 테이블
lookup(3 - 2*a + b - c, output) in  chi_base_table
where  output == a ^ ((~b) & c)
```

테스트 전략

단위 테스트

```
#[test]
fn  test_keccak_simple() {
// 알려진 입력/출력 쌍 테스트
let  input = b"";
let  expected = "c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a47
// ...
}
```

```
#[test]
fn  test_keccak_multi_block() {
// 멀티 블록 입력 테스트
let  input = vec![0u8; 200];
// ...
```

```
}
```

통합 테스트

```
#[test]
fn test_keccak_circuit() {
let k = 19;
let inputs = vec![
vec![],
b"hello".to_vec(),
vec![0u8; 200],
];
let circuit = KeccakCircuit::new(2usize.pow(k), inputs);
let prover = MockProver::run(k, &circuit, vec![]).unwrap();
assert_eq!(prover.verify(), Ok(()));
}
```

관련 문서

이전/다음

- **이전**: Bytecode Circuit - 바이트코드 검증
- **다음**: Tx Circuit - 트랜잭션 검증

같은 섹션 (핵심 회로)

- EVM Circuit - EVM 실행 검증
- State Circuit - MPT 노드 해시 검증
- Bytecode Circuit - 코드 해시 검증에 Keccak 사용
- Tx Circuit - 트랜잭션 해시 계산
- Sig Circuit - 서명 검증
- Copy Circuit - 복사 연산

관련 테이블

- Keccak Table - 테이블 스키마 상세
- 테이블 시스템 - 테이블 개요
- Lookup 메커니즘 - 룩업 동작

고급 회로 (다른 해시 함수)

- SHA256 Circuit - SHA256 해시
- Poseidon Circuit - Poseidon 해시

최적화 기법

- CellManager - 수직 할당 전략
- RLC 기법 - Random Linear Combination
- 병렬화 - 병렬 처리

구현 세부사항

- Halo2 통합 - Halo2 API
- 제약 조건 구축 - ConstraintBuilder

- Gadget 시스템 - 재사용 가능한 컴포넌트

참고 자료

주요 파일:

- `zkevm-circuits/src/keccak_circuit.rs` : 회로 설정 및 게이트
- `zkevm-circuits/src/keccak_circuit/keccak_packed_multi.rs` : 핵심 알고리즘 구현
- `zkevm-circuits/src/keccak_circuit/cell_manager.rs` : Keccak 전용 CellManager
- `zkevm-circuits/src/keccak_circuit/param.rs` : 파라미터 정의
- `zkevm-circuits/src/keccak_circuit/util.rs` : 유틸리티 함수
- `zkevm-circuits/src/table.rs` : KeccakTable 정의

핵심 함수:

- `multi_keccak()` : 여러 입력에 대한 witness 생성
- `keccak()` : 단일 입력 처리
- `pack()` / `unpack()` : Sparse representation 변환
- `field_xor()` : 필드 원소 XOR 연산
- `get_num_bits_per_lookup()` : 동적 part size 계산

핵심 구조체:

- `KeccakCircuitConfig<F>` : 회로 설정
- `KeccakRow<F>` : 행 witness 데이터
- `CellManager<F>` : 셀 할당 관리
- `AbsorbData<F>` : Absorb 데이터
- `SqueezeData<F>` : Squeeze 데이터

환경 변수:

- `KECCAK_DEGREE` : 회로 높이 (기본값: 19)
- `KECCAK_ROWS` : 라운드당 행 수 (기본값: 12)

외부 참고:

- Keccak 공식 명세
- FIPS 202 표준
- Ethereum Yellow Paper: Keccak-256 사용